

Heterogeneous Reflex Reasoning: Where Multi-Engine Interface Knowledge Breaks

Working Draft

August 2026

Abstract

Where should knowledge of a deterministic execution interface live: in context, in model weights, or in the runtime? We test one bounded protocol spanning five local engines—calculator, Datalog, ISA graph, date arithmetic, and dimensional unit conversion—on Qwen3-0.6B, SmolLM2-1.7B, and Gemma3-1B. A frozen benchmark contains 200 training, 50 development, and 100 test rows, with 16 test tasks per engine and 20 no-execution controls. A prompt-derived deterministic reflex and an oracle both execute the frozen forms exactly. The learned policy does not: at five engines, Qwen, SmolLM2, and Gemma reach engine-selection rates of 0.188, 0.350, and 0.000, respectively, and runtime-exact rates of 0.188, 0.313, and 0.000. All preserve zero false activations, but every exact result sent through the Qwen or SmolLM2 result-use pass is overridden. In-context demonstrations recover some engine families but grow from 39 to 118 static lexical interface tokens and reach false-activation rates of 0.9–1.0 at several catalog sizes. Textual schemas remain safe but inert. A follow-up factorization adds 2,250 train, 450 development, and 900 test rows. Its oracle-engine deterministic parser is exact. A benchmark-specific lexical router reaches 0.900 engine selection and runtime exactness with zero false activation, whereas semantic rules select 0.800 but execute 1.000 by substituting graph closure for Datalog, and learned CPU routers select only 0.588 with 0.400 false activation. Thus exact execution and parsing belong in the runtime, but broad semantic selection and result use remain unresolved. Factorized Qwen router LoRAs improve selection to 0.848 (attention only) and 0.900 (attention+MLP), but false activation rises to 0.273 and 0.400; neither clears the gate. The Paper 3 gate remains `no_go`.

1 Question and decision rule

Paper 1 established a narrow placement rule for one calculator interface: stable selection and serialization may live in adapter weights, while parsing, bounds, execution, provenance, reinjection, and enforcement remain deterministic. This paper asks whether that rule survives heterogeneous interfaces. The central question is:

Can one small model learn when to offload, select among five local engines, serialize a bounded typed request, and preserve the exact result without prompt cost growing with the engine catalog?

The confirmatory gate is intentionally stronger than demonstrating that the engines work. At five engines, at least two model families must reach both engine-selection and runtime-exact rates of at least 0.8, with false activation at most 0.1. Deterministic or oracle success cannot open this gate.

2 Bounded heterogeneous runtime

2.1 Common envelope and registry

All engines use one fail-closed registry and normalize into shared typed request, result, and append-only state records. The runtime accepts exactly one complete fenced block from an enabled family. Open fences, multiple blocks, unknown engines, invalid dimensions, malformed payloads, and over-budget state fail without execution.

The five block families are:

```
```calculator          ```date
15246377 * 746647383 add 2026-08-28 P90D
```                   ```
```

```

```datalog
fact link(a,b)
fact link(b,c)
query reachable(a,c)
```

```graph
isa penguin bird
isa bird animal
query isa penguin animal
```

```units
convert 7.3 mile -> kilometer
```

```

The calculator is bounded and exact; Datalog uses finite forward chaining; the graph engine computes bounded ISA closure and frame inheritance; date arithmetic uses ISO dates and integer day durations; unit conversion rejects dimensional mismatch. No generated code, shell command, network action, general planner, or unbounded theorem proving is permitted.

2.2 Runtime reflex versus oracle

The *runtime reflex* recognizes only the five anchored natural-language templates frozen by the benchmark and translates their captured fields into a typed block. Unknown or modified input returns no execution. It does not read the gold target. The separate *oracle* condition replays the gold typed envelope. Both are deterministic protocol validations, not model evidence and not claims of open-ended semantic routing.

3 Experimental design

3.1 Frozen data and leakage audit

Table 1 summarizes the benchmark. Train, development, and test use disjoint operand ranges, entity prefixes, dates, values, and control labels. A machine-readable audit finds no train/development/test signature overlap and no answer string in a typed training target. Targets teach requests, never engine answers.

Table 1: Frozen benchmark composition.

| Family | Train | Dev | Test |
|----------------------|-------|-----|------|
| Calculator | 32 | 8 | 16 |
| Datalog | 32 | 8 | 16 |
| ISA graph | 32 | 8 | 16 |
| Date/time | 32 | 8 | 16 |
| Units | 32 | 8 | 16 |
| No-execution control | 40 | 10 | 20 |
| Total | 200 | 50 | 100 |

Nested catalogs contain 1, 2, 3, and 5 engines in the order calculator, Datalog, graph, date, and units. Each catalog includes all 20 controls. This changes the positive-task denominator (16, 32, 48, and 80) while holding the control set fixed.

3.2 Placement conditions

We compare seven roles:

1. unassisted base-model exact answers;
2. context/ICL with one demonstration per enabled engine;
3. explicit textual schemas, because these checkpoints do not provide one uniform reliable native-tool interface;

4. one five-family rank-8 LoRA adapter plus a 26-token stable contract;
5. the anchored deterministic runtime reflex;
6. learned-policy plus deterministic validation/execution;
7. oracle typed-envelope replay.

ICL and explicit-schema generation are tested on Qwen across all four catalog sizes. Learned-weight scaling uses the same five-family Qwen adapter while the runtime enables nested catalogs. Five-engine learned replication includes all three model families. Result use is assessed in the full five-engine adapter runs; scaling sweeps stop after execution to avoid conflating selection with a second generation pass.

3.3 Adapters and XPU execution

Each adapter trains for two epochs on the same 200 request-policy rows, with 50 development rows, rank 8, alpha 16, dropout 0.05, batch size 1, gradient accumulation 8, learning rate 2×10^{-4} , and projections **q/k/v/o**. Final development losses are 0.00380 (Qwen), 0.00564 (SmolLM2), and 0.01122 (Gemma). Low teacher-forced loss is therefore not treated as evidence of held-out generation reliability.

All checkpoint revisions are pinned: Qwen **c1899de**, SmolLM2 **31b70e2**, and Gemma **dcc83ea**. Training and generation use PyTorch 2.13.0+XPU on Intel Graphics. Interface generations have a 72-token cap; the uniformly short unassisted exact-answer controls use a 24-token cap.

3.4 Factorized scoring

For positive rows we score DETECT, ENGINE SELECT, PAYLOAD NORMALIZE, EXECUTE, runtime exactness, REINJECT, USE, and FINAL independently. False activation is defined only on controls. USE, ignored-result, override, and wrong- reinterpretation rates are conditioned on rows for which result use was actually assessed. Aggregate final accuracy includes controls and is reported only with that caveat. Costs include static lexical interface tokens, model tokenizer prompt and generated tokens, reinjection tokens, model and engine calls, CPU engine time, XPU wall time, and typed-state bytes/items.

3.5 Post-gate diagnostic benchmark

After the confirmatory failure, we freeze a larger diagnosis rather than tune against the original test. It contains 250 training, 50 development, and 100 test positives per engine, plus 1,000/200/400 controls: 2,250/450/900 rows in total. Split-specific paraphrase frames and entity namespaces are disjoint; graph and Datalog truth labels are balanced; audits find no duplicate IDs, cross-split prompt overlap, or answer tokens in request targets. Arithmetic now includes three operations, dates include offset and difference, units span length/mass/time, and controls contain engine vocabulary without requesting execution.

We separate six-way classification from payload construction. T0 is the old anchored parser, T1 lexical/regex routing, T2 per-engine semantic parsing, T3 a word-ngram TF-IDF logistic classifier, and T4 a 48-dimensional TF-IDF/SVD classifier. Every selected engine feeds the same deterministic per-engine payload parser and bounded runtime. An oracle engine label with that parser is the serialization upper bound. The semantic gate requires at least 0.9 engine selection and runtime exactness with false activation at most 0.1; T1 is retained separately as a benchmark-specific deployment baseline.

4 Results

4.1 Five-engine outcome

Table 2 gives the central result. The bounded reflex and oracle are exact, but no learned model approaches the gate. Unassisted positive accuracy is zero for every family/model cell under strict exact matching. The apparent unassisted final scores of 0.10–0.20 come only from controls.

The adapters fail differently. Qwen detects every positive and every control, but collapses calculator, Datalog, graph, and units toward malformed Datalog-like blocks. Its only exact family is date (15/16). SmolLM2 normalizes units at 16/16 and date at 9/16, with all other families at zero. Gemma produces no exact typed request. Table 3 makes the concentration visible.

Table 2: Five-engine results. Selection, exactness, and use are positive-row rates; FAR is over controls; final includes controls. A dash means use was not assessed.

| Placement | Model | Select | Runtime exact | FAR | Use | Final |
|-------------------|---------------|--------|---------------|-------|-------|-------|
| Runtime reflex | Deterministic | 1.000 | 1.000 | 0.000 | 1.000 | 1.000 |
| Oracle | Deterministic | 1.000 | 1.000 | 0.000 | 1.000 | 1.000 |
| Unassisted | Qwen3-0.6B | 0.000 | 0.000 | 0.000 | – | 0.100 |
| Unassisted | SmolLM2-1.7B | 0.000 | 0.000 | 0.000 | – | 0.200 |
| Unassisted | Gemma3-1B | 0.000 | 0.000 | 0.000 | – | 0.200 |
| ICL | Qwen3-0.6B | 0.475 | 0.475 | 1.000 | – | 0.380 |
| Textual schemas | Qwen3-0.6B | 0.000 | 0.000 | 0.000 | – | 0.200 |
| Weights + runtime | Qwen3-0.6B | 0.188 | 0.188 | 0.000 | 0.000 | 0.200 |
| Weights + runtime | SmolLM2-1.7B | 0.350 | 0.313 | 0.000 | 0.000 | 0.200 |
| Weights + runtime | Gemma3-1B | 0.000 | 0.000 | 0.000 | – | 0.200 |

Table 3: Exact executions out of 16 held-out rows per family for five-engine learned adapters.

| Model | Calculator | Datalog | Graph | Date | Units |
|--------------|------------|---------|-------|------|-------|
| Qwen3-0.6B | 0 | 0 | 0 | 15 | 0 |
| SmolLM2-1.7B | 0 | 0 | 0 | 9 | 16 |
| Gemma3-1B | 0 | 0 | 0 | 0 | 0 |

4.2 Post-gate factorization: routing, not execution

Table 4 isolates the failure. With the gold engine, the deterministic parser obtains 1.000 payload and runtime exactness and zero control activation. T1 reaches the numerical threshold but depends directly on the benchmark lexicon. T2 identifies four of five positive engine families. It maps Datalog reachability to graph closure, so payload and engine identity score 0.800 while all positive answers remain runtime-exact. This is useful engine substitutability, not correct protocol selection.

Table 4: CPU trigger-ladder diagnosis on 500 positives and 400 controls. Payload and runtime are positive-row rates; FAR is over controls.

| Condition | Select | Payload exact | Runtime exact | FAR | Mean route μ s |
|------------------------|--------|---------------|---------------|-------|--------------------|
| T0 anchored parser | 0.000 | 0.000 | 0.000 | 0.000 | 0.7 |
| T1 lexical/regex | 0.900 | 0.900 | 0.900 | 0.000 | 4.6 |
| T2 semantic rules | 0.800 | 0.800 | 1.000 | 0.000 | 9.3 |
| T3 TF-IDF linear | 0.588 | 0.588 | 0.688 | 0.400 | 13.1 |
| T4 latent CPU | 0.588 | 0.588 | 0.688 | 0.400 | 14.3 |
| Oracle engine + parser | 1.000 | 1.000 | 1.000 | 0.000 | – |

Shallow lexical audits do not solve the six-way task: bag-of-words, word-ngram, character-ngram, and latent classifiers obtain 0.633, 0.593, 0.773, and 0.593 accuracy, respectively. Their best positive engine-selection rate is 0.592. For word TF-IDF, increasing examples per engine from 25 to 250 raises trigger recall from 0.570 to 0.988 but also raises FAR from 0 to 0.400; selection ends at 0.588. More rows therefore expose a decision-boundary problem rather than repairing it. The next justified neural experiment is factorized six-way routing, not another joint free-form block generator.

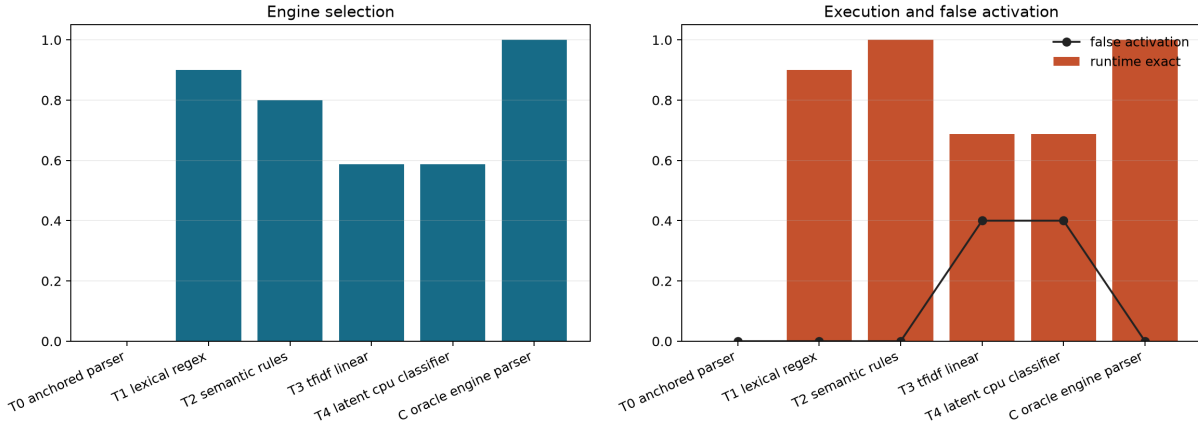


Figure 1: Six-way routing accuracy and end-to-end runtime exactness. The oracle parser bounds serialization; T1 is lexical and T2 exploits graph/Datalog semantic equivalence rather than selecting every engine correctly.

4.3 Factorized neural routing and LoRA targets

The CPU diagnosis justifies one T5 experiment: train Qwen3-0.6B to emit only one of the six engine labels, then pass that label to the same deterministic per-engine parser. The prompt, data hashes, rank 8, alpha 16, dropout 0.05, two-epoch schedule, seed, and pinned base revision are fixed. L1 adapts q/k/v/o; L2 also adapts gate/up/down. The unadapted base emits NONE on all 900 rows. Table 5 shows that factorization helps substantially but does not clear the safety gate.

Table 5: Factorized router comparison on the same 500 positives and 400 controls. Route latency is per row; training is XPU wall time.

| Condition | Select | Runtime exact | FAR | Route ms | Train min | Trainable |
|-----------------------|--------|---------------|-------|----------|-----------|-----------|
| T1 lexical/regex | 0.900 | 0.900 | 0.000 | 0.0047 | 0 | 0 |
| T2 semantic rules | 0.800 | 1.000 | 0.000 | 0.0071 | 0 | 0 |
| Qwen base | 0.000 | 0.000 | 0.000 | 344.5 | 0 | 0 |
| Qwen L1 attention | 0.848 | 0.948 | 0.273 | 514.2 | 67.4 | 2.29M |
| Qwen L2 attention+MLP | 0.900 | 1.000 | 0.400 | 554.7 | 82.7 | 5.05M |

L1 is exact on all calculator and graph rows and 99/100 dates; it routes 75/100 units correctly and splits Datalog 50/50 between Datalog and the answer-equivalent graph closure. It activates on 109/400 controls. L2 fixes calculator, date, units, and Datalog, but splits graph 50/50 into Datalog and activates on 160/400 controls. Of its 900 outputs, 80 control completions are malformed and fail closed; hence parse-valid rate is 0.911. Functional engine substitution explains why L2 runtime exactness reaches 1.000 despite 0.900 engine identity. It does not excuse the control failures.

L2 trains 5.05M parameters (0.840%) and peaks at 1.42 GiB XPU allocation, versus 2.29M (0.383%) and 1.38 GiB for L1. Its development loss is slightly lower (0.0851 versus 0.0897–0.0917), but held-out FAR is worse. Thus lower teacher-forced loss and more adapter capacity do not imply safer routing. The machine-readable decision retains T1 for this frozen language and advances to result-use factorization rather than another joint block generator.

4.4 Capability-count scaling and interference

Figure 4 and Table 6 show that recurring context grows with the catalog, but quality is not monotonic. ICL is exact on all calculator positives at one engine and all calculator/Datalog positives at two, yet its false-activation rates are 1.0 and 0.9. Adding graph drops positive exactness to 0.396; at five engines exactness is 0.475 and FAR returns to 1.0. Textual schemas always emit no execution: safe, increasingly expensive, and inert. Qwen weights

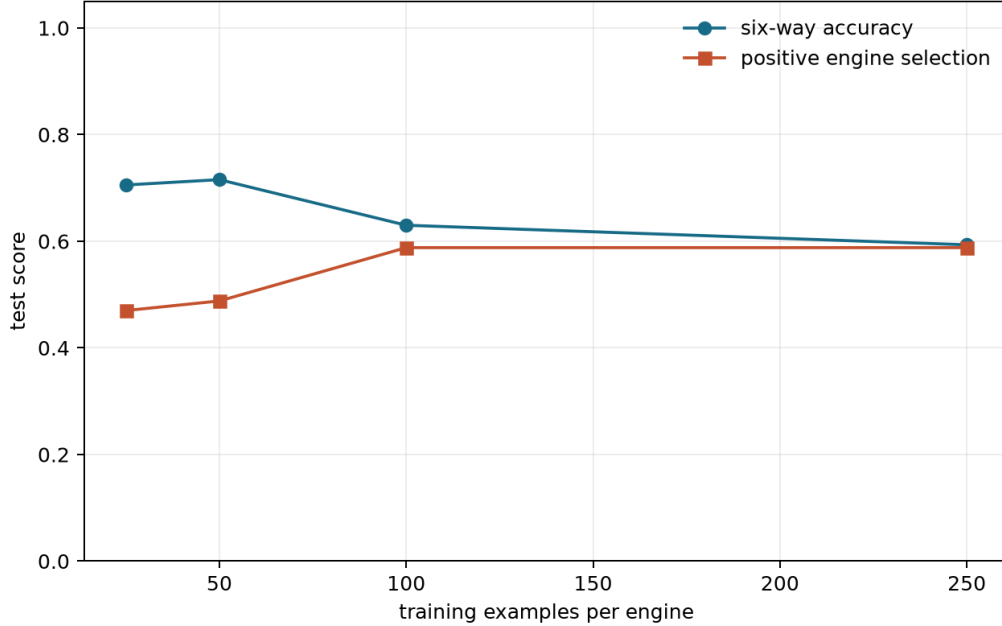


Figure 2: Word TF-IDF does not approach reliable six-way selection as the training set grows; increasing trigger recall is accompanied by control activation.

keep a constant 26-token contract and FAR 0, but exactness is zero at 1–3 engines and 0.188 at five because date rows enter the test mix.

4.5 Exact execution is not exact result use

The second model pass receives the original question and the deterministic result, then must answer with that result only. Qwen receives 15 such passes and SmolLM2 receives 28. Both have use accuracy 0.0 and override rate 1.0 among assessed rows. Wrong reinterpretation occurs in 11/15 Qwen passes and all 28 SmolLM2 passes. Thus execution success is not sufficient: result preservation is a separate failure surface that the runtime must eventually enforce.

4.6 Typed state and bounded composition

Forty deterministic two-stage compositions validate dependency-linked state: 20 date $\rightarrow$ calculator and 20 graph $\rightarrow$ Datalog rows. Both families achieve 1.0 at stage 1, stage 2, final exactness, dependency recording, and state reuse. Mean state sizes are 3 and 7 items, respectively. This shows that the typed runtime can reuse derived values without repeated model calls; it does not show that a model can plan those compositions.

4.7 Prototype economics

Mean deterministic reflex wall time is below 1 ms at every catalog size on this machine. Model conditions range from about 0.7 s per row for an inert one-engine schema prompt to 13.4 s for the five-engine SmolLM2 adapter. These are correctness-first, full-prefix prototype measurements on one Intel GPU, not production latency estimates. The stronger conclusion is operational: engine CPU time is negligible relative to neural generation, but selecting and preserving the engine result remains unreliable.

5 Hypotheses, falsification, and gate

- H1 (reliable heterogeneous learned interface) is falsified at this scale.
- H2 (context burden grows with engine count) is supported for recurring tokens, but constant adapter context does not yield reliable quality.

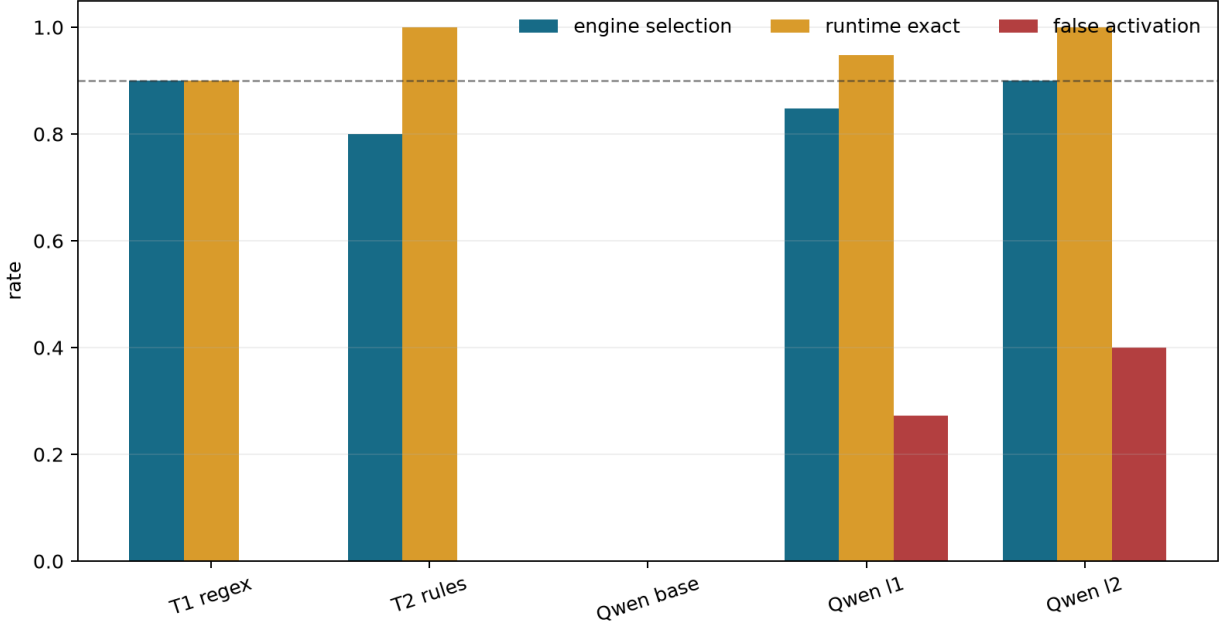


Figure 3: Factorized routing improves positive selection, but neither neural adapter meets the joint selection/runtime/FAR gate. The dashed line marks the 0.9 positive-rate threshold; FAR must independently remain at most 0.1.

- H3 (assistance degrades more slowly with task difficulty) is not adjudicated; the frozen test varies values, not formal depth.
- H4 (robustness to added engines) is falsified by non-monotonic ICL interference and learned family collapse.
- H5 (typed-state reuse) is supported only in deterministic bounded compositions.
- H6 (small-model augmentation closes a large-model gap) is not tested; no larger-model comparison is used to rescue a failed small-model gate.
- H7 (deterministic serialization is the dominant failure) is falsified by the exact oracle-engine parser; semantic routing is the remaining upstream bottleneck, with result use remaining downstream.
- H8 (factorized LoRA clears the semantic route gate) is falsified for Qwen: L1 misses selection and FAR, while L2 reaches selection but worsens FAR.

The machine-readable gate artifact records **status: no_go** and an empty passing-model list. Paper 3 may still study implicit semantic interrupts as a new exploratory question, but this explicit multi-engine result does not justify claiming a reliable learned predecessor.

The post-gate diagnostic does not reverse this decision. T1 is the cheapest passing baseline for the frozen language, but no T2–T4 semantic route clears the selection criterion. Consequently Paper 3 remains paused while factorized result-use controls are tested. The completed Qwen L1/L2 comparison also fails the gate, so it does not reverse that pause.

6 Limitations

The benchmark is synthetic and deliberately templated. Disjoint namespaces and a frozen test reduce leakage, but do not establish linguistic robustness. The runtime reflex is anchored to those templates and should not be generalized. Only one seed, one rank, one training schedule, and three small checkpoints are tested. The explicit baseline uses textual schemas rather than native function calling. Strict exact matching penalizes semantically correct capitalization or extra prose, appropriately for an execution protocol but more harshly than a question-answer benchmark. We do not add the optional sixth engine, a generic code interpreter, an unrestricted planner,

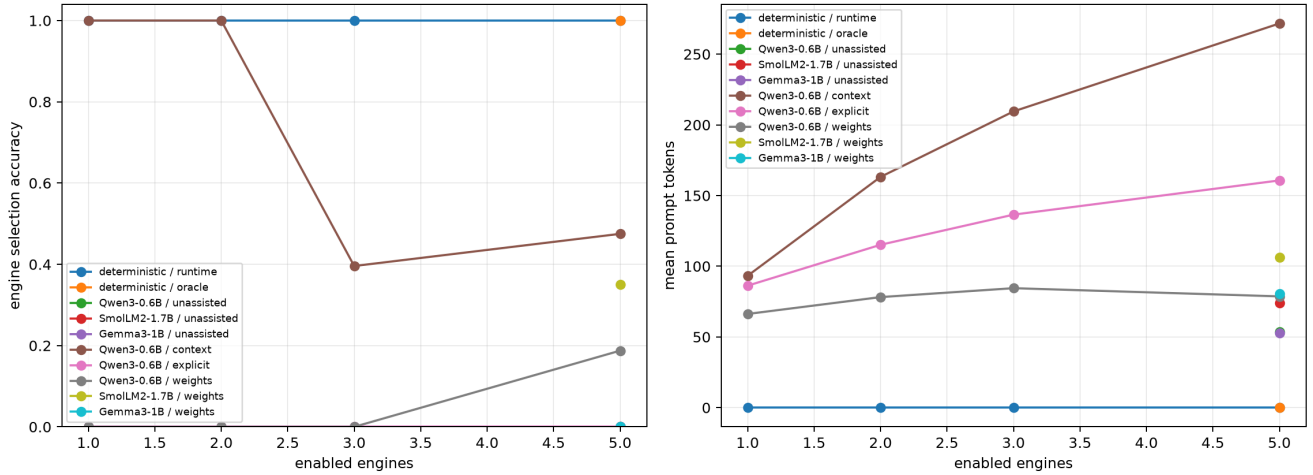


Figure 4: Engine selection and mean tokenizer prompt cost versus enabled engine count. Deterministic and oracle lines are protocol bounds, not learned evidence.

larger models, or a model-authored composition study after the core gate fails. Finally, measured XPU wall time is backend- and machine-specific.

The larger diagnosis is still synthetic and has only split-level template separation, not naturally authored language. T1 should therefore not be read as open-domain robustness. Graph and Datalog deliberately share a transitive closure subproblem; T2’s runtime success shows functional substitution but also why answer-only scoring would conceal routing errors.

The factorized neural comparison uses one checkpoint family and one seed. It tests attention-only versus attention+MLP targets, not layer-region or per-engine adapter selection. These results diagnose Qwen3-0.6B at this scale; they do not establish that every neural router must fail. Conversely, the 0.5-second XPU route cost is machine-specific and should not be generalized as production latency.

A post-study TwIL-LM3/SmolLM3-3B interface pilot is retained in the repository but excluded from the headline results. Its Horn and graph labels were all positive, admitting a constant-**true** neural baseline, and its **/no_think** 160-token budget does not match TwIL’s documented thinking-mode evaluation. Strict rescoring does establish one useful boundary: every exact IR executed correctly, while only 6/22 SmolLM3 and 7/22 TwIL hybrid exact tasks had both the right formalization and result. These are interface diagnostics, not a ranking of internal reasoning. A balanced, TwIL-aligned revision is required before adding “reasoning in weights versus coprocessors” as a main result.

7 Reproducibility

The retained workflow is:

```
python -m ccpu paper2 generate-next \
  --config configs/paper2/next_iter_xpu.json \
  --output-dir artifacts/paper2/next_iter/data_v1
```

```
python -m ccpu paper2 run-next \
  --config configs/paper2/next_iter_xpu.json \
  --dataset artifacts/paper2/next_iter/data_v1/test.jsonl \
  --condition runtime --catalog-size 5 \
  --output-dir artifacts/paper2/next_iter/runtime_n5_v4
```

```
python -m ccpu paper2 analyze-next \
  --config configs/paper2/next_analysis.json \
  --output-dir artifacts/paper2/next_iter/analysis_v1
```


Table 6: Qwen capability scaling. Static is lexical interface burden; prompt is the measured mean model-token count.

| Condition | Engines | Static | Prompt | Select/exact | FAR | Mean wall ms |
|--------------|---------|--------|--------|--------------|-------|--------------|
| ICL | 1 | 39 | 93.3 | 1.000 | 1.000 | 2709 |
| ICL | 2 | 62 | 163.2 | 1.000 | 0.900 | 9846 |
| ICL | 3 | 92 | 209.5 | 0.396 | 0.200 | 4383 |
| ICL | 5 | 118 | 271.7 | 0.475 | 1.000 | 10033 |
| Text schemas | 1 | 43 | 86.3 | 0.000 | 0.000 | 705 |
| Text schemas | 2 | 56 | 115.2 | 0.000 | 0.000 | 819 |
| Text schemas | 3 | 69 | 136.5 | 0.000 | 0.000 | 957 |
| Text schemas | 5 | 95 | 160.7 | 0.000 | 0.000 | 1130 |
| Weights | 1 | 26 | 66.3 | 0.000 | 0.000 | 2479 |
| Weights | 2 | 26 | 78.2 | 0.000 | 0.000 | 6696 |
| Weights | 3 | 26 | 84.5 | 0.000 | 0.000 | 8689 |
| Weights | 5 | 26 | 78.7 | 0.188 | 0.000 | 7843 |

```
python -m ccpu paper2 generate-diagnostic \
--config configs/paper2/diagnostic_cpu.json \
--output-dir artifacts/paper2/diagnostic_v1/data
```

```
python -m ccpu paper2 audit-diagnostic \
--train artifacts/paper2/diagnostic_v1/data/train.jsonl \
--test artifacts/paper2/diagnostic_v1/data/test.jsonl \
--output artifacts/paper2/diagnostic_v1/lexical_audit.json
```

```
python -m ccpu paper2 analyze-diagnostic \
--train artifacts/paper2/diagnostic_v1/data/train.jsonl \
--test artifacts/paper2/diagnostic_v1/data/test.jsonl \
--output-dir artifacts/paper2/diagnostic_v1/analysis
```

```
python -m ccpu paper2 prepare-router \
--source-dir artifacts/paper2/diagnostic_v1/data \
--output-dir artifacts/paper2/router_v1/data
```

```
python -m ccpu paper2 train-next-lora \
--config configs/paper2/router_qwen_l1_xpu.json \
--model qwen3_0_6b --train artifacts/paper2/router_v1/data/train.jsonl \
--dev artifacts/paper2/router_v1/data/dev.jsonl \
--output-dir artifacts/paper2/router_v1/lora_qwen_l1_v1
```

```
python -m ccpu paper2 run-router \
--config configs/paper2/router_qwen_l1_xpu.json --model qwen3_0_6b \
--dataset artifacts/paper2/router_v1/data/test.jsonl \
--condition T5_qwen_router_l1 \
--adapter-path artifacts/paper2/router_v1/lora_qwen_l1_v1/adapter \
--output-dir artifacts/paper2/router_v1/qwen_l1_v1
```

```
python -m ccpu paper2 analyze-router \
--config configs/paper2/router_analysis.json \
```

`--output-dir artifacts/paper2/router_v1/analysis`

The original model runs store predictions, factorized summaries, dataset and artifact hashes, pinned model metadata, package/device provenance, and the dirty Git revision at execution time. The earlier analysis artifact hashes all 22 source summaries. The CPU diagnostic manifests record split/source hashes, configuration, Python/platform/Git environment, predictions, and aggregate reports.

8 Related work

Toolformer learns model-authored API use [1]; this study instead holds execution local, typed, and fail-closed. LoRA supplies the compact weight placement mechanism [2], but low adapter loss does not guarantee exact interface generation. AlphaGeometry combines neural guidance and symbolic deduction in a domain-specific system [3]. Dynamic solver composition and neurosymbolic primitive programming study richer solver selection and composition [4, 5]; our negative result is narrower and focuses on factorized interface reliability in small models.

9 Conclusion

The answer to “where should interface knowledge live?” is asymmetric. Exact parsing, execution, provenance, state, and enforcement belong in the runtime. Context can expose heterogeneous interfaces but scales in recurring tokens and can activate when it should not. Adapter weights keep the contract compact and controls safe, but the tested adapters collapse across engine families and fail to preserve exact returned values. The missing component is therefore not another exact engine; it is a reliable, independently validated semantic selection and result-use policy. Until that component clears a multi-family held-out gate, heterogeneous learned reflex reasoning remains unconfirmed. The larger factorization narrows the next step: keep deterministic per-engine serialization, retain lexical routing as a bounded baseline, and train/evaluate semantic selection independently before spending capacity on generation. For the tested Qwen router, increasing LoRA target breadth does not solve that selection problem safely; runtime result enforcement remains the next cheapest factorized intervention.

References

- [1] Schick et al. Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS, 2023.
- [2] Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. ICLR, 2022.
- [3] Trinh et al. Solving Olympiad Geometry without Human Demonstrations. Nature, 2024.
- [4] Xu et al. Adaptive LLM-Symbolic Reasoning via Dynamic Logical Solver Composition. EACL, 2026.
- [5] Bhat et al. Forethought: Verifiable Reasoning from Neurosymbolic Primitive Programming. arXiv:2607.04096, 2026.